Санкт-Петербургский государственный университет

Козенко Дмитрий Сергеевич

Выпускная квалификационная работа

Ускорение запуска тестов на `Verilog` симуляторе

Уровень образования: бакалавриат

Направление *02.03.03 «Математическое обеспечение и администрирование информационных систем»*

Основная образовательная программа *СВ.5162.2020 «Технологии программирования»*

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н., Гориховский В.И.

Рецензент:
должность, ООО «Место работы», степень (если есть), Рецензент Р. Р.

Санкт-Петербург
2026

Saint Petersburg State University

Kozenko Dmitriy

Bachelor's Thesis

Accelerating test execution in a `Verilog` simulator

Education level: bachelor

Speciality *02.03.03 "Software and Administration of Information Systems"*

Programme *CB.5162.2020 "Programming Technologies"*

Scientific supervisor:
docent of the Chair of System Programming, C.Sc., V.I. Gorihovskiy

Reviewer:
position at "Company", degree if present R.R. Reviewer

Saint Petersburg
2026

Оглавление

Введение **4**

1. Обзор **5**

1.1. Отложенное форматирование 6

1.2. Инструменты и подходы для ускорения симуляции . . . 9

1.3. Форматирование на стороне симулятора 10

1.4. Выводы 11

2. Постановка задачи **13**

3. Решение **14**

4. Заключение **15**

Список литературы **16**

Введение

Эта задача является актуальной...

1. Обзор

Принятые сокращения:

- *ТУ* (*DUT*) — Тестируемое Устройство (Device Under Test).
- *УРП* (*RTL*) — Уровень Регистровых Передач (Register-Transfer Level).
- *ФАПЧ* (*PLL*) — Фазовая Автоподстройка Частоты (Phase-Locked Loop).
- *СнК* (*SoC*) — Система на Кристалле (System on a Chip).

Отдельный тест ТУ на `Verilog`-симуляторе может идти часами и сутками. При этом важно тестировать именно УРП. Сложную логику, например криптографию, лучше тестировать на более быстрых платформах, например `QEMU` [2]. А при тестировании отдельных частей УРП вместо честной симуляции других частей их можно эмулировать. Например, достаточно тестировать работу с регистрами `UART` или же `flash`-памяти в отдельных тестах, а в других, где их функциональность нужна только для обеспечения общей логики теста, их работу можно эмулировать, так как кроме общего ускорения теста это может быть критически важно по причине того, что объем тестируемого ввода и вывода может быть настолько крупным, что будет занимать на порядки больше времени, чем реально полезная работа по тестированию нужного УРП. Но даже с учетом этих значительных ускорений всё равно остаётся актуальным вопрос общего ускорения времени работы теста. Компиляторы симуляторов уже умеют выбрасывать лишнюю логику, распараллеливать модель и отключать трассировку. Эта работа рассматривает более узкое место — форматированный вывод, который ТУ делает по ходу теста. Даже несмотря на то, что функциональность `UART` для вывода байта эмулируется, форматирование вывода и его посимвольный вывод занимают много времени на ТУ, так как каждая операция процессора выполняется в разы дольше, чем на хосте.

Ключевая идея — перенести логику форматирования строки вывода с ТУ на хост, где эта операция относительно ТУ будет бесплатна, при этом добиться минимального количества процессорных инструкций на ТУ.

1.1. Отложенное форматирование

Существуют различные подходы к решению данной задачи, основная часть которых заключается в особой обработке макросов вывода для создания уникальной быстро распознаваемой структуры вывода, которая переводится в изначально заложенный вывод отдельным приложением. В общем случае эта структура выглядит следующим образом — уникальный идентификатор строчки форматирования (например, в `ELF`-файле или уникальный хеш, считающийся на этапе компиляции), а после этого аргументы в бинарном формате. Чтобы ТУ напечатало сообщение с N аргументами форматирования, при такой структуре нужно $O(N)$ операций с небольшой константой. Это направление ускорения вывода можно назвать *отложенным форматирование*.

Способ вывода аргументов зависит от вида канала.

Если канал бинарный, то можно напрямую копировать аргументы побайтно, что позволяет обрабатывать их за несколько процессорных операций на ТУ. Если же канал текстовый, то аргументы необходимо кодировать.

Для перевода бинарных данных в текстовый формат существует общепринятая кодировка `base64`, описанная в стандарте `RFC 4648` [4], но данная операция занимает много процессорных операций. Альтернативный способ, работающий лучше `base64` на маленьких объемах данных (а аргументы форматирования как раз такие — за исключением `long double`, все аргументы занимают не более 8 байт) — вывод в шестнадцатеричном виде. Но в итоге для текстового канала выигрыш достаточно незначительный относительно бинарного.

Библиотеки и системы с реализацией отложенного форматирования обладают рядом недостатков.

- Повышенная сложность кода, усложняющая поддержку, так как для достижения максимального уменьшения процессорных операций на ТУ используют дополнительные проходы сборки и кодогенерацию для обработки макросов вывода.

- Необходимость переводить вывод ТУ в читаемый для человека вид через отдельные инструменты, что усложняет запуск программы для ручного использования и затрудняет автоматизацию тестирования, особенно если требуется разбор текста в реальном времени.

Ниже перечислены примеры библиотек, предоставляющих данную функциональность.

1.1.1. Библиотека `NanoLog`

Библиотека `NanoLog` реализует отложенное форматирование для серверных `C++`-программ [11, 10]. Механизм идеально подходит под приведенное выше общее описание отложенного форматирования. Во время компиляции из вызова функции вывода вырезается статическая часть сообщения, во время исполнения остаётся только вывод компактной бинарной записи. Отдельный процесс после прогона подставляет аргументы и получает обычный текст. Но целевой сценарий использования у этой библиотеки другой — ведение журналов в низколатентных серверных программах на `C++`. Требуются максимально быстрая обработка запроса и высокая производительность журналирования — вывод распознаётся позже (при резервном копировании или при возникновении ошибки). Накладные расходы на вывод сообщения находятся в порядке наносекунд. Благодаря этому достигается пропускная способность в десятки миллионов сообщений в секунду, на один-два порядка выше, чем у библиотек журналирования с прямым форматированием `spdlog` и `log4j`. Далее пропускная способность упирается в задержку системных вызовов `x86` для обращения к журналу.

1.1.2. Операционная система реального времени Zephyr

В операционной системе реального времени (RTOS) Zephyr для встроенных систем есть отдельная подсистема отложенного форматирования, называемая системным журналом [12]. Во время сборки все статические строчки (в том числе и с форматом) заменяются на ссылки в ELF — для этого требуется дополнительный проход сборки, при котором специальная программа собирает по всему проекту макросы вывода и составляет словарь log_dictionary.json, сопоставляющий адрес в ELF-файле с изначальной строкой. Словарь после пересборки инвалидируется, а без актуального словаря вывод восстановить невозможно. Во время выполнения программы выводится идентификатор строчки и аргументы в бинарном представлении (для текстовых каналов можно сконфигурировать перевод бинарного набора данных в шестнадцатеричный вид).

Поддерживается распознавание вывода как в режиме реального времени, так и после завершения запуска.

1.1.3. Библиотека Pigweed

Pigweed — это набор библиотек от компании Google для встроенных программ. В этом наборе есть модуль pw_tokenizer, который встраивается в уже существующий журнал [6].

Строчки форматирования преобразуются в 4-байтовый хеш на этапе компиляции при помощи constexpr-функции для C++ или заранее сгенерированных макросов для C и на этапе компоновки собираются в отдельную секцию ELF-файла, по которой в дальнейшем создаётся словарь для перевода вывода в изначальный вид.

Вывод на ТУ во время исполнения создаёт бинарный пакет, в начале которого лежит созданный хеш строки форматирования, а далее аргументы, упакованные следующим образом.

- Целочисленные аргументы упаковываются через varint [3].
- Числа с плавающей точкой пишутся в бинарном формате побайтно.

- Строчки (для спецификатора форматирования `%s`) копируются полностью.

Как и в `Zephyr`, для текстовых каналов можно сконфигурировать кодирование в текстовый формат перед отправкой, но в данном случае уже `base64`.

1.2. Инструменты и подходы для ускорения симуляции

Как уже было сказано в начале обзора, при тестировании ТУ на конкретной части УРП другие части можно не симулировать, а эмулировать, чтобы ускорить время прохождения теста. Ниже приведены несколько примеров, когда это разумно применяется.

- Работу с регистрами `UART` можно проверять в отдельном тесте, после этого можно выводить байт сразу, обращаясь к симулятору.
- После тестирования системы тактирования вместо честной настройки ФАПЧ для СнК можно настроить подаваемую на конкретное тестируемое устройство частоту, запросив ее у симулятора напрямую, так как ФАПЧ — сложное устройство, симуляция которого создает большую нагрузку на симулятор.
- После проверки работы с регистрами устройства `flash`-памяти даже на относительно небольших данных можно запрашивать у симулятора доставку данных из `flash` в память ТУ за одну команду.
- Для остановки симуляции с возвратом кода ошибки также требуется возможность взаимодействовать с симулятором из ТУ.

Для реализации механизма управления симуляцией из ТУ существуют различные подходы, некоторые из них описаны ниже.

- `Semihosting` [7, 1] — протокол, позволяющий ТУ использовать ресурсы и данные хостового компьютера: после особой последовательности машинных команд ТУ, среди которых есть команда точки останова, ТУ приостанавливается, симулятор выполняет запрошенную операцию (конфигурируемую через регистр `a0` ТУ и аргумент этой операции в регистре `a1`), а после этого продолжает исполнение ТУ.

- Ловушка системного вызова — похожий на `semihosting` механизм, при котором ТУ помещает аргументы в определенные регистры, а потом вызывает `ecall`, а хост перехватывает этот системный вызов ТУ и выполняет запрошенное действие.

- Отображенный в память ТУ блок регистров (Memory-Mapped Interface, `MMI` или Memory-Mapped I/O, `MMIO`) — симулятор отслеживает запись в определенную область памяти и реагирует на нее. Например, на запись по определенному адресу читает значение оттуда и печатает его на экран — эмуляция работы `UART`, описанная выше.

1.3. Форматирование на стороне симулятора

Как уже было сказано в разделе 1.1, отложенное форматирование обладает определенным списком недостатков, хотя и позволяет вынести форматирование за пределы ТУ, при этом никак не меняя симулятор.

Но если встроить в симулятор поддержку вывода форматированных строк, можно достичь значительного ускорения — печать сообщений никак не влияет на проверку УРП отдельных устройств, за исключением `UART`. Используя один из механизмов, описанных в разделе 1.2, можно предоставить ТУ возможность быстрого вывода сообщений в режиме реального времени без значительного усложнения тестируемого кода.

Подобным образом работает библиотека `SEGGER Semihosting` [8]. Она предоставляет возможность форматирования и вывода сообщения на хосте: от ТУ требуется минимальное количество операций —

собрать аргументы форматирования в `va_list` и передать в функцию `SEGGER_SEMIHOST_Writef` адрес строки с форматом и этот список аргументов.

Листинг 1: Обёртка `printf` через `SYS_WRITEF`

```
int myprintf(const char *s, ...) {
    va_list List;
    va_start(List, s);
    SEGGER_SEMIHOST_Writef(s, &List);
    va_end(List);
    return 0;
}
```

Кроме этого есть свежая работа от разработчиков компании `Google` [5], которая описывает схожий механизм, но основанный уже не на `semihosting`, а на блоке `MMIO`-регистров. Авторы только описывают примененную идею и не прилагают никаких программных реализаций — это внутренняя разработка компании.

1.4. Выводы

Были рассмотрены два основных подхода к переносу логики форматирования из ТУ на хост — отложенное форматирование и форматирование на стороне симулятора.

Отложенное форматирование обладает рядом определённых неудобств, описанных в разделе 1.1. Есть готовые библиотеки для его интеграции, которые упрощают поддержку кода, но трудности остаются: интеграция, увеличение времени компиляции и преобразование вывода в изначальный вид.

Форматирование на стороне симулятора лишено этих недостатков, но готовых программных решений в индустрии не представлено — реализация этой функциональности, несмотря на простоту относительно полноценной системы отложенного форматирования, находится глубоко в кодовой базе отдельного симулятора.

Целевой симулятор VCS [9] по умолчанию не поддерживает обработку команд semihosting. Кроме того, semihosting создаёт задержку из-за полной остановки исполнения кода на ТУ, что также отрицательно влияет на прирост от форматирования на хосте. Поэтому интеграция библиотеки SEGGER Semihosting [8] также может быть не лучшим решением для достижения максимальной производительности.

Наиболее подходящий по скорости работы, простоте интеграции и дальнейшего использования подход описан в работе разработчиков компании Google [5], так как целевой симулятор уже обладает блоком MMI-регистров, который можно легко расширить. При этом данный подход лишён недостатков отложенного форматирования и будет работать быстрее semihosting.

2. Постановка задачи

Целью работы является ...

Для достижения поставленной выше цели необходимо выполнить следующие задачи.

1. ...

3. Решение

Было сделано то-то

4. Заключение

В ходе работы были достигнуты следующие результаты.

- Задача 1 выполнена

Список литературы

[1] Arm Limited. Semihosting for AArch32 and AArch64. — Application Binary Interface for the Arm Architecture, release 2025Q4. — 2026. — https://github.com/ARM-software/abi-aa/blob/main/semihosting/semihosting.rst.

[2] Bellard Fabrice. QEMU, a Fast and Portable Dynamic Translator // Proceedings of the 2005 USENIX Annual Technical Conference. — Anaheim, CA : USENIX Association, 2005. — P. 41–46.

[3] Google. Encoding. — Protocol Buffers Documentation. — 2024. — https://protobuf.dev/programming-guides/encoding/.

[4] Josefsson Simon. RFC 4648: The Base16, Base32, and Base64 Data Encodings. — Internet Engineering Task Force. — 2006. — https://www.rfc-editor.org/rfc/rfc4648.

[5] Keller Daniel, Kumar Rohit. Stop Paying by the Letter: Accelerating Full-Chip RTL Simulation with Host-Side Tokenized Logging. — arXiv:2610.27356. — 2026. — https://arxiv.org/abs/2610.27356.

[6] Pigweed. pw_tokenizer: Compress Strings to Shrink Logs. — Pigweed Documentation. — 2026. — https://pigweed.dev/pw_tokenizer/.

[7] RISC-V International. RISC-V Semihosting Specification. — Version 1.0, ratified. — 2025. — https://docs.riscv.org/reference/semihosting/_attachments/riscv-semihosting.pdf.

[8] SEGGER. SEGGER Semihosting. — SEGGER Knowledge Base. — 2024. — https://kb.segger.com/SEGGER_Semihosting.

[9] Synopsys. Verification Continuum VCS User Guide. — Version V-2023.12. — 2023. — Руководство поставляется с лицензией симулятора.

[10] Yang Stephen. NanoLog: A Nanosecond Scale Logging System : Ph. D. thesis / Stephen Yang ; Stanford University. — 2020. — URL: https://web.stanford.edu/~ouster/cgi-bin/papers/YangPhD.pdf.

[11] Yang Stephen, Park Seo Jin, Ousterhout John. NanoLog: A Nanosecond Scale Logging System // 2018 USENIX Annual Technical Conference (USENIX ATC 18). — Boston, MA : USENIX Association, 2018. — P. 335–350. — URL: https://www.usenix.org/conference/atc18/presentation/yang-stephen.

[12] Zephyr Project. Dictionary-Based Logging. — Zephyr Project Documentation. — 2026. — https://docs.zephyrproject.org/latest/services/logging/index.html.